

Lec12

Javascript Introduction

JavaScript (JS) is a high-level, interpreted, and dynamic programming language used to make web pages interactive and responsive. It is one of the core technologies of web development, alongside **HTML** and **CSS**.

- **HTML** provides the structure of a webpage.
- **CSS** provides styling and layout.
- **JavaScript** adds behavior and interactivity.

Javascript can

Validate form inputs

Create animations

Respond to user actions (clicks, keyboard input, etc.)

Update webpage content without reloading

Communicate with servers and APIs

Features of Javascript

Lightweight and Interpreted

JavaScript code is executed directly by the browser without requiring compilation.

Dynamic Typing

Variables can store different types of data without specifying their type.

```
let x = 10;  
x = "Hello";
```

Object-Oriented

JavaScript supports objects, classes, and inheritance.

Event-Driven

It can respond to user actions such as clicks, mouse movements, and keyboard events.

Cross-Platform

JavaScript runs on almost all modern web browsers and operating systems.

Client-Side and Server-Side

- Client-side: Runs in the browser.
- Server-side: Runs on servers using platforms like Node.

Variables in Javascript

Variables are used to store data values that can be used and modified during program execution.

Declaring variables

var

```
var name = "Naman";
```

- Function-scoped
- Can be redeclared and updated

let

```
let age = 21;
```

- Block-scoped
- Can be updated but not redeclared in the same scope

const

```
const PI = 3.14159;
```

- Block-scoped
- Cannot be reassigned after initialization

Data Types in Javascript

Primitive Data Types

1. Number

```
let x = 100;  
let y = 10.5;
```

2. String

Text enclosed in quotes.

```
let name = "Naman";
```

or

```
let city = 'Delhi';
```


Boolean

Stores true or false.

```
let isStudent = true;  
let isLoggedIn = false;
```

4. Undefined

Variable declared but not assigned a value.

```
let x;  
console.log(x);
```

Output:

undefined

5. Null

Represents an empty value.

```
let data = null;
```

Non-Primitive Data Types

Object

```
let student = {  
  name: "Naman",  
  age: 21  
};
```

Array

```
let colors = ["Red", "Blue", "Green"];
```

Function

```
function greet() {  
  console.log("Hello");  
}
```

Operators in Javascript

Arithmetic Operators

Operator	Meaning	Example
+	Addition	5+3
-	Subtraction	5-3
*	Multiplication	5*3
/	Division	5/3
%	Modulus	5%3
**	Exponent	2**3

Assignment Operators

`let x = 10;`

`x += 5; // x = x + 5`

`x -= 2; // x = x - 2`

`x *= 3; // x = x * 3`

`x /= 2; // x = x / 2`

Comparison Operators

Operator	Meaning
==	Equal value
===	Equal value and type
!=	Not equal
!==	Not equal value or type
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Logical Operators

Operator	Meaning
&&	AND
	OR
!	NOT

Increment and Decrement

```
let x = 5;
```

```
x++;
```

```
x--;
```

```
console.log(x);
```

Type Casting in Javascript

Type casting (Type Conversion) means converting one data type into another.

There are two types:

1. Implicit Type Casting
2. Explicit Type Casting

Implicit Type Casting

JavaScript automatically converts data types.

Example:

```
let result = "10" + 5;
```

```
console.log(result);
```

Output:

105

Explanation:

- Number 5 is automatically converted to string.

ANother Example

```
let result = "10" - 5;
```

```
console.log(result);
```

Output:

5

JavaScript converts "10" to number.

Explicit Type Casting

Programmer manually converts data types.

Convert String to Number

Number()

```
let str = "100";  
let num = Number(str);
```

```
console.log(num);
```

Output:

100

parseInt()

```
let x = parseInt("50");
```

Output:

50

parseFloat()

```
let y = parseFloat("10.5");
```

Output:

10.5

Convert Number to String

```
let num = 100;  
let str = String(num);
```

```
console.log(str);
```

Output:

"100"

Convert to Boolean

```
console.log(Boolean(1));  
console.log(Boolean(0));
```

Output:

true

false

Conditions in Javascript

Conditions are used to make decisions in a program based on whether an expression is true or false.

if Statement

```
let age = 18;
```

```
if (age >= 18) {  
    console.log("You can vote");  
}
```

Output:

You can vote

if-else Statement

```
let age = 16;
```

```
if (age >= 18) {  
    console.log("You can vote");  
} else {  
    console.log("You cannot vote");  
}
```

Output:

You cannot vote

if-else if-else Ladder

```
let marks = 85;
```

```
if (marks >= 90) {  
    console.log("Grade A+");  
} else if (marks >= 75) {  
    console.log("Grade A");  
} else if (marks >= 60) {  
    console.log("Grade B");  
} else {  
    console.log("Grade C");  
}
```

Output:

Grade A

switch Statement

Used when multiple conditions depend on the same variable.

```
let day = 3;
```

```
switch(day) {  
  case 1:  
    console.log("Monday");  
    break;  
  case 2:  
    console.log("Tuesday");  
    break;  
  case 3:  
    console.log("Wednesday");  
    break;  
  default:  
    console.log("Invalid Day");  
}
```

Output:

Wednesday

Loops in Javascript

Loops execute a block of code repeatedly.

for Loop

```
for(let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

while Loop

```
let i = 1;

while(i <= 5) {
  console.log(i);
  i++;
}
```

Output:

```
1
2
3
4
5
```

do-while Loop

Executes at least once.

```
let i = 1;

do {
  console.log(i);
  i++;
} while(i <= 5);
```

Output:

```
1
2
3
4
5
```

for...of Loop

Used to iterate through arrays.

```
let fruits = ["Apple", "Banana", "Mango"];
```

```
for(let fruit of fruits) {  
  console.log(fruit);  
}
```

Output:

```
Apple  
Banana  
Mango
```

for...in Loop

Used to iterate through object properties.

```
let student = {  
  name: "Naman",  
  age: 21  
};
```

```
for(let key in student) {  
  console.log(key, student[key]);  
}
```

Output:

```
name Naman  
age 21
```

Scope in Javascript

Global Scope

Variables declared outside all functions and blocks.

```
let name = "Naman";
```

```
function show() {  
  console.log(name);  
}
```

```
show();
```

Output:

Naman

The variable can be accessed anywhere.

Function Scope

Variables declared with `var` inside a function.

```
function demo() {  
  var x = 10;  
  console.log(x);  
}
```

```
demo();
```

Outside the function:

```
console.log(x);
```

Output:

ReferenceError

Block Scope

Variables declared using `let` and `const`.

```
if(true) {  
  let x = 10;  
  const y = 20;  
  
  console.log(x);  
  console.log(y);  
}
```

Outside the block:

```
console.log(x);
```

Output:

ReferenceError

Scope Comparison

Keyword	Global Scope	Function Scope	Block Scope
var	Yes	Yes	No
let	Yes	Yes	Yes
const	Yes	Yes	Yes

Basic Problem-Solving in JavaScript

Example 1: Check Even or Odd

Example 2: Find Largest of Two Numbers

Example 3: Sum of First N Numbers

Example 4: Multiplication Table

Example 5: Count Digits in a Number

Task

Solve pattern-based and number-based JavaScript problems

N=5

```
1                      9
2 8                      8 2
3 7 3                    7 3 7
4 6 4 6                  6 4 6 4
5 5 5 5 5 5 5 5 5 5 5
6 4 6 4                  4 6 4 6
7 3 7                    3 7 3
8 2                      2 8
9                          1
```